

Prova tecnica — Sviluppatore Junior Full-Stack

Costruisci una piccola applicazione web per la gestione del noleggio di DVD. L'obiettivo non è la completezza assoluta, ma il modo in cui modelli il problema e scrivi il codice.

Stack

Il nostro team lavora con due stack; scegli quello con cui ti trovi meglio:

- **ASP.NET Core (C#) + React**
- **Laravel (PHP) + React** (con **Inertia.js** opzionale)

Valutiamo come usi lo stack, non quale scegli.

Contesto

«Sono il gestore di un negozio che noleggia film in DVD. Ho un catalogo di titoli e una clientela affezionata, e ogni giorno registro noleggi e restituzioni. Vorrei un'applicazione web che mi permetta di gestire comodamente i noleggi dei miei DVD: tenere d'occhio cosa è disponibile, sapere chi ha preso cosa e segnare i rientri.»

Modello dati

Entità	Campi	Note
Cliente	nome, cognome, email	Creato e gestito via API .
DVD	titolo, data di uscita, categoria, durata (min), quantità	Catalogo precaricato via seed .
Noleggio	cliente, DVD, data noleggio, restituzione prevista, restituzione effettiva	La restituzione effettiva è valorizzata al rientro del DVD.

Requisiti funzionali

1. **Clienti** — un endpoint per creare un cliente e uno per elencarli.
2. **Catalogo** — visualizzare l'elenco dei DVD (popolabile via seed), con il numero di **copie disponibili** (es. 2 di 3).

3. **Noleggio** — registrare il noleggio di un DVD con copie disponibili da parte di un cliente, impostando data di noleggio e restituzione prevista.

Facoltativi (apprezzati ma non indispensabili):

4. **Restituzione** — chiudere un noleggio: la copia torna tra quelle disponibili.
 5. **Storico** — consultare i noleggi, con possibilità di filtrare per cliente e titolo del DVD.
-

Definition of Done

- L'app gira in locale seguendo il README, i requisiti richiesti sono implementati.
-

Cosa valutiamo

- **Approccio al problema** — come affronti la risoluzione del problema e la logica adottata.
 - **Leggibilità** — nomi sensati, codice pulito, niente over-engineering.
 - **Coerenza** — il flusso funziona davvero, dall'interfaccia al database.
 - **UI** — interfaccia funzionale e ordinata.
-

Bonus (facoltativi)

Avvio via Docker · seguire un pattern architetturale · gestione mobile (responsive).

Consegna

Hai **una settimana di tempo** dalla ricezione della traccia per consegnare il progetto. Conta più la qualità della completezza: meglio poco e fatto bene che molto e disordinato.

Pubblica il codice su un repository Git e condividi il link, con un **README** che contenga:

- lo stack scelto;
- come avviare backend, frontend e seed;
- eventuali scelte o assunzioni rilevanti.

Per qualsiasi dubbio scrivici: anche **fare domande sensate** è parte di ciò che apprezziamo. In bocca al lupo!